Generalized Sudoku with SAT: Comparing Three At-Most-One Encodings and CP/ILP Baselines

Tạ Đình Nguyên[1]

Modern Problems in Computer Science, INT3301E_4, semester I, 2026–2027

ARTICLE INFO

Keywords:
Sudoku
SAT
At-Most-One encoding
constraint programming
integer linear programming

ABSTRACT

This report evaluates three At-Most-One encodings for a cell–value Boolean model of generalized Sudoku, alongside CP-SAT, HiGHS, and commercial solver baselines. Beyond a separately reported three-repetition baseline, the expanded analysis covers 16,000 puzzles and 110,000 primary jobs in one repetition: 109,664 checker-passed SAT outcomes and 336 non-completions. Runtime summaries use only stated SAT intersections; the main encoding panel uses 86 common-SAT N25 puzzles from one resource-contended 32-worker session. Results show encoding and profile-specific runtime tradeoffs; this report does not estimate repeatability across repetitions, confidence intervals, or isolated speedups. A valid witness does not verify uniqueness, and synthetic puzzles share roots rather than acting as independent samples.

1. Introduction and related work

Sudoku has concise rules, yet the same feasibility problem can be represented in SAT, CP, or ILP. As grid size increases, representations can differ in variable counts, constraints, and model construction costs. Evaluation therefore needs to distinguish representation size, construction/loading time, and solving time; fewer clauses alone do not establish faster solving. This work considers generalized Sudoku with $n \geq 2$ and $N = n^2$, focusing on three AMO encodings within a common SAT formulation.

1.1. Related foundations

Lynce–Ouaknine formulate Sudoku using cell–value Boolean variables and distinguish minimal and extended models [10]. Their work supplies modeling background, not results for this project's dataset. Sinz presents a sequential unary counter for cardinality constraints [13]. PySAT supplies a toolkit and the CardEnc API for AMO choices [9, 11]. Here, ALO and the four EXO families stay fixed while AMO varies among pairwise, seqcounter, and bitwise; bitwise AMO is not identified with Sinz's binary counter. These sources support construction methods, not correctness checks or measurements of the current program.

1.2. Research questions

1. How do the three AMO encodings affect primary/auxiliary variables, clauses, and build/solve/total times as grid size and clue density vary?
2. On matching instance sets, how does SAT compare with CP-SAT and HiGHS ILP in valid-solution coverage and time under the same cutoff?
3. What additional evidence do available Gurobi/CPLEX comparisons on the small subset provide, and how do runtime/license limits restrict coverage?

S80/B16/C10 specify comparison subsets before measurement. Section 12 answers these questions using checked measurements and the correct intersections; SAT statistics for all of S80 are not combined with B16/C10 baseline statistics as though they used the same sample.

1.3. Scope of the contribution

The course-project contribution is an implementation of generalized Sudoku models, integration of established encodings under a common specification, model/solution validation, and evaluation through a 906-run/three-repetition baseline plus an expanded analysis of 16,000 puzzles and 110,000 primary jobs in one repetition. Expanded results retain SAT intersections and bound inference by profile, denominator, and repetition. No new encoding is proposed, existing runtimes/libraries are not presented as algorithms invented by the author, and no superiority is claimed for all n or all Sudoku instances.

Result scope

The report separates the three-repetition baseline from the expanded one-repetition analysis. Each expanded table and figure states its denominator, data intersection, and interpretive limits; runtimes include only checker-passed SAT outcomes. The snapshot, receipt, analysis contract, and hashed exports are retained locally as described in Code and data availability.

[1]Student ID: 24021587. Individual course project 1. Instructor: Dr. Tô Văn Khánh.

2. Problem definition and notation

Let $n \geq 2$ be an integer and $N = n^2$. The $N \times N$ Sudoku grid contains N blocks of size $n \times n$. Rows and columns are zero-based; cell values belong to $D = \{1, \dots, N\}$. Input uses 0 for an empty cell. A solution completes the grid, preserves its clues, and contains each value exactly once in every row, column, and block. Uniqueness is not required.

Define $I = \{0, \dots, N-1\}$, $J = \{0, \dots, n-1\}$, and the nonzero clue triples $G \subseteq I \times I \times D$, with $g = |G|$. The Boolean variable $x_{r,c,v}$ is true exactly when cell (r, c) contains value v. There are N^3 primary variables. The implementation maps them to

$$\mathrm{id}(r, c, v) = (rN + c)N + v \ \in \{1, \dots, N^3\}. \tag{1}$$

The parser rejects malformed structure or domains as INVALID_INPUT. Structurally valid conflicting clues are still encoded; infeasibility becomes UNSAT only when proved. Figure 1 illustrates $n = 2$.

1			4
	4	1	
2		4	
	3		1

→

1	2	3	4
3	4	1	2
2	1	4	3
4	3	2	1

Figure 1: A 4×4 input (left) and a constructed completion (right), for illustration; not solver output or a uniqueness certificate.

3. SAT formulation and three AMO encodings

The cell–value representation and extended formulation of Lynce–Ouaknine provide background [10]. For a group $Z = (z_1, \dots, z_m)$, Exactly-One decomposes into ALO and AMO:

$$\mathrm{EXO}(Z) = \mathrm{ALO}(Z) \wedge \mathrm{AMO}(Z), \qquad \mathrm{ALO}(Z) = \bigvee_{i=1}^{m} z_i. \tag{2}$$

The following four families each contain N^2 groups of length N. Together with the clue units they form the Sudoku formula:

$$\bigwedge_{r,c \in I} \mathrm{EXO}\{x_{r,c,v} : v \in D\}, \qquad \text{cell;} \tag{3}$$

$$\bigwedge_{r \in I, v \in D} \mathrm{EXO}\{x_{r,c,v} : c \in I\}, \qquad \text{row–value;} \tag{4}$$

$$\bigwedge_{c \in I, v \in D} \mathrm{EXO}\{x_{r,c,v} : r \in I\}, \qquad \text{column–value;} \tag{5}$$

$$\bigwedge_{p,q \in J, v \in D} \mathrm{EXO}\{x_{pn+u,qn+w,v} : u, w \in J\}, \qquad \text{block–value;} \tag{6}$$

$$\bigwedge_{(r,c,v) \in G} x_{r,c,v}, \qquad \text{clues.} \tag{7}$$

Thus there are $4N^2$ EXO groups. Code uses the same group/literal order for every encoder, appends ALO before AMO within each group, and then appends clues in row-major order. Duplicate clauses remain; no symmetry breaking or clue-specific preprocessing is applied.

3.1. Pairwise

$$E_{\mathrm{pw}}(Z) = \bigwedge_{1 \leq i < j \leq m} (\neg z_i \vee \neg z_j). \tag{8}$$

Every input pair is forbidden from being simultaneously true. Code generates these pairs directly without auxiliaries.

3.2. Sequential counter

Following the sequential unary counter principle [13], AMO can be specified using a chain $s_1, \dots, s_{m-1}$:

$$E_{\mathrm{sc}}(Z, S) = (\neg z_1 \vee s_1) \wedge (\neg z_m \vee \neg s_{m-1}) \wedge \bigwedge_{i=2}^{m-1} \big[(\neg z_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg z_i \vee \neg s_{i-1})\big]. \tag{9}$$

A true input propagates along the chain, excluding any later true input. For an extension, choose $s_i = \bigvee_{j \leq i} z_j$. The program calls `CardEnc.atmost` with `bound=1`, `EncType.seqcounter`, and the current `top_id` [11, 9]. The expression specifies the principle and semantics; it does not claim byte-for-byte literal order or auxiliary naming in the PySAT binary.

3.3. Bitwise AMO

Let $k = \lceil \log_2 m \rceil$ and assign each input a distinct binary code $b(i) \in \{0,1\}^k$. With selectors $a_1, \dots, a_k$, let $\ell_{ij} = a_j$ if $b(i)_j = 1$, and $\ell_{ij} = \neg a_j$ otherwise. Then

$$E_{\text{bw}}(Z, A) = \bigwedge_{i=1}^{m} \bigwedge_{j=1}^{k} (\neg z_i \vee \ell_{ij}). \tag{10}$$

Two true inputs require distinct codes and therefore conflict on at least one bit. With one true input, choose its code; with no true inputs, selectors are arbitrary. The program uses `EncType.bitwise` through the same AMO API. This encodes AMO over one-hot literals; it does not replace Sudoku cell variables by binary numbers. The bitwise API/source is credited separately [11, 12], not attributed to Sinz.

4. Correctness

Let $F_e(X, A)$ be the CNF for encoding e, including clues. Each AMO must satisfy $\exists A_Z \, E_e(Z, A_Z) \iff \sum_i z_i \le 1$, as the constructions above do. Code reserves all N^3 primary IDs first, allocates each group's auxiliaries above `top_id`, and updates it, so groups do not share auxiliaries. Thus

$$\exists A \, F_e(X, A) \quad \iff \quad \text{Sudoku}_G(X). \tag{11}$$

Forward direction. Cell ALO and AMO give exactly one value per cell. Row–value, column–value, and block–value groups require every value exactly once in the corresponding unit. Unit clauses preserve every clue. Decoding a CNF model's primary variables therefore yields a valid solution.

Reverse direction. From a Sudoku solution preserving clues, set the selected cell–value variables true. Every group has exactly one true input. Pairwise is already satisfied; extend sequential groups by prefix ORs and bitwise groups by the true input's code. Disjoint auxiliary sets let these local extensions combine into a global satisfying assignment A. This does not require every auxiliary assignment to satisfy the CNF, nor imply uniqueness.

5. Theoretical size and checked counts

For $m \ge 4$, the presented AMO constructions have the following auxiliary counts $a_e(m)$ and clause counts $c_e(m)$:

e	$a_e(m)$	$c_e(m)$
pairwise	0	$m(m-1)/2$
seqcounter	$m-1$	$3m-4$
bitwise	$\lceil \log_2 m \rceil$	$m\lceil \log_2 m \rceil$

(12)

Including one ALO per group, the theoretical totals are

$$V_e = N^3 + 4N^2 a_e(N), \qquad C_e = 4N^2[1 + c_e(N)] + g. \tag{13}$$

Pairwise has exactly N^3 variables and $4N^2[1 + N(N-1)/2] + g$ clauses with duplicates retained. Clause growth for the constructions is respectively $\Theta(N^4)$, $\Theta(N^3)$, and $\Theta(N^3 \log N)$; all have $\Theta(N^3)$ total variables, since $g \le N^2$. These describe representation size, not solve-time bounds or evidence of a faster encoder.

Table 1 is generated from `diagnostics/model/results.json`: PySAT 1.9.dev16, $N \in \{4, 9, 16, 25\}$. These are pre-preprocessing CNF counts, not benchmarks. Diagnostic cases have $g = 2$; the table subtracts the two clue units to show $C - g$. Observed seqcounter/bitwise counts match the expressions at these four sizes. This does not assert identical counts for every variant, library, or input length; small groups may use API shortcuts. The upstream source read is not treated as proof of the running binary.

Table 1
Structural counts from diagnostics; $C - g$ excludes clue units.

N	AMO	N^3	Aux.	V_e	$C_e - g$
4	pairwise	64	0	64	448
4	seqcounter	64	192	256	576
4	bitwise	64	128	192	576
9	pairwise	729	0	729	11988
9	seqcounter	729	2592	3321	7776
9	bitwise	729	1296	2025	11988
16	pairwise	4096	0	4096	123904
16	seqcounter	4096	15360	19456	46080
16	bitwise	4096	4096	8192	66560
25	pairwise	15625	0	15625	752500
25	seqcounter	15625	60000	75625	180000
25	bitwise	15625	12500	28125	315000

6. CP and ILP comparison models

The comparison models describe the same task: finding one completion that preserves the clues, not optimizing puzzle difficulty or solution quality. This section describes the accepted builders; successful construction does not establish successful solving or license coverage at every size.

6.1. CP with AllDifferent

Using the previously defined I, J, D, G, introduce an integer variable $z_{r,c} \in D$ for each cell and impose

$$\text{AllDifferent}(z_{r,c} : c \in I), \qquad r \in I, \tag{14}$$

$$\text{AllDifferent}(z_{r,c} : r \in I), \qquad c \in I, \tag{15}$$

$$\text{AllDifferent}(z_{pn+u,qn+w} : u, w \in J), \qquad p, q \in J, \tag{16}$$

$$z_{r,c} = v, \qquad (r, c, v) \in G. \tag{17}$$

Each unit has N cells with an N-element domain, so AllDifferent forces each value to occur exactly once. There are N^2 integer variables and $3N + g$ model-level constraints; these are not clause/variable counts after internal solver transformations.

The CP-SAT builder creates a `CpModel`, a fresh `CpSolver`, and sets one worker [3]. The CP Optimizer builder uses the same domains, AllDifferent constraints, and clues with `Workers=1` [8]; its executable is checked separately. The accepted builder package does not itself solve, decode, or validate a solution.

6.2. Common binary ILP

Let $y_{r,c,v} \in \{0, 1\}$ for $r, c \in I$ and $v \in D$. Use four equality families and the clues:

$$\sum_{v \in D} y_{r,c,v} = 1, \qquad r, c \in I, \tag{18}$$

$$\sum_{c \in I} y_{r,c,v} = 1, \qquad r \in I,\ v \in D, \tag{19}$$

$$\sum_{r \in I} y_{r,c,v} = 1, \qquad c \in I,\ v \in D, \tag{20}$$

$$\sum_{u,w \in J} y_{pn+u,qn+w,v} = 1, \qquad p, q \in J,\ v \in D, \tag{21}$$

$$y_{r,c,v} = 1, \qquad (r, c, v) \in G. \tag{22}$$

The builder exports a CSR matrix M, an all-ones right-hand side, and an all-zero objective:

$$\min\ \mathbf{0}^{\mathrm{T}} Y = 0 \quad \text{subject to} \quad MY = \mathbf{1}, \quad 0 \le Y \le 1, \quad Y \in \mathbb{Z}^{N^3}. \tag{23}$$

All feasible solutions have objective 0, so this is a feasibility problem. The objective does not minimize execution time. Zero-based columns use $\mathrm{id}(r, c, v) - 1$. The model has N^3 binary variables, $4N^2 + g$ equality rows, and $4N^3 + g$ nonzeros before preprocessing. Each EXO group/clue produces one row; potentially redundant rows remain, with no backend-specific domain reduction or pruning.

The same ILP structure is used by Gurobi, CPLEX MIP, and HiGHS adapters. Execution, thread/budget settings, statuses, and checker integration are described separately in Section 7; builder evidence alone does not establish runtime coverage. At the model level, $y_{r,c,v} = 1 \iff z_{r,c} = v$ yields the same Sudoku solutions as the SAT representation, despite different internal solver representations.

7. Algorithm and adapter implementation

The pseudocode below summarizes the implemented parser, model builder, solver adapter, decoder, and separate checker. Earlier equations specify the models; the actual source code belongs in the repository. This workflow does not replace a runner with a hard process guard or establish that a benchmark has run.

7.1. Library use and validation

The SAT backend is `pysat.solvers.Glucose3` in python-sat 1.9.dev16 [9, 1, 2]. The five baselines use OR-Tools CP-SAT [3], gurobipy [4], the direct CPLEX Python API (`cplex.Cplex`, without DOcplex.MP) [7], DOcplex.CP for CP Optimizer [8], and highspy for HiGHS [5, 6]. In these adapters SciPy supplies sparse ILP matrices; it is not a HiGHS solve backend through `scipy.optimize.milp`. For SAT, the public API consists of `prepare_backend` and `solve_sat`; each call uses a fresh Glucose3 instance. The adapter calls the CNF builder with the selected encoding, uses interrupt-capable native solving, then passes the model through decoding and `validation.check_solution`. Auxiliary variables do not determine decoded cell values.

For comparisons, `solve_baseline` accepts one of `cpsat`, `gurobi`, `cplex_mip`, `cpoptimizer`, or `highs`. CP-SAT/CP Optimizer use the CP builders; Gurobi/CPLEX MIP/HiGHS consume the same ILP matrix, bounds, integrality, and constant

Algorithm 1 Solve one Sudoku and validate its result

1. Read/parse the input into a Puzzle; structural, JSON, or file errors produce INVALID_INPUT. Check method/encoding/budget against the API before building.
2. Prepare backend libraries outside measurement; report missing runtime as UNAVAILABLE.
3. Start the monotonic clock at t_0; use the earlier of t_0+cutoff and any caller deadline. If the budget is exhausted, return TIMEOUT.
4. Build the CNF or CP/ILP model and create/load a fresh solver. Account for build/setup; if the budget is exhausted, skip solving and return TIMEOUT.
5. Call the solver with the remaining budget, using supported native limits and cooperative interruption. Retain the raw status.
6. Record native-solve completion and observed component times. Late results become TIMEOUT; do not admit a late incumbent.
7. Return UNSAT only on proved infeasibility. With no acceptable candidate, classify TIMEOUT, ERROR, or LICENSE_LIMIT from evidence; do not turn unknown into UNSAT.
8. Decode the candidate into a complete grid; decoding failure is ERROR. SAT decodes primary variables only; MIP values must pass finite/integrality checks before selection.
9. Run the independent checker on domains, rows, columns, blocks, and clues. Publish SAT with a solution only if accepted; rejection or checker failure is ERROR.
10. On every exit path, cancel/join timers before native solver deletion and release resources. Report cleanup failure explicitly rather than publishing a successful solution.

objective. Each call has fresh objects and one thread/worker. Adapters retain raw statuses and use the same production checker.

MIP decoding accepts only finite values within 10^{-6} of 0 or 1, with exactly one selected value per cell; arbitrary rounding of fractional solutions is not allowed. HiGHS must also report acceptable primal feasibility/integrality before decoding. A time-limit incumbent can become SAT only if it arrived on time and passes checking. UNSAT requires infeasibility evidence; missing runtime, license limits, and unknown outcomes remain distinct.

7.2. Timing boundary and process safeguards

Imports/input reading precede measurement. Build includes model/CNF construction and solver creation/loading; solving uses the remaining budget. Decoding, checking, and cleanup are outside build+solve. Deadline admission uses native-solve completion, not checker completion as the solve endpoint. Timing components unknown after forced termination must remain missing rather than invented.

SAT native interruption and baseline timers/native limits are cooperative. In DOcplex 2.32.264, TimeLimit is embedded in transmitted CPO, so an exact post-load native TimeLimit is not promised: the adapter uses preload for build accounting, an abort_search timer against the actual deadline, and late-result rejection. The private preload hook needs rechecking on DOcplex upgrades. If timer joining fails, the adapter reports ERROR and may defer native deletion to avoid a callback into released memory.

The measurement-end hook and hard worker/process-tree guard are accepted. The runner uses START/END to apply the cutoff to measurement, then switches to a postprocessing watchdog and process-tree cleanup as needed. The pilot exercised this integration; native timeout alone remains cooperative. Section 9 describes the frozen main configuration; Section 12 reports its executed results separately from the pilot.

8. Expanded-experiment protocol

Table 2
Target denominator by configuration; not observed coverage.

N	Target puzzles	Configurations	Primary runs
9	10,000	8	80,000
16	5,000	5	25,000
25	1,000	5	5,000
Total	16,000		110,000

The table multiplies puzzle quotas by eligible configurations by N for one repetition. It is not an observed count. ERROR/TIMEOUT remain primary coverage rows; inapplicable configurations create no job. Final results distinguish planned, selected, observed, and solved denominators.

The extension matrix fixes three sizes: public N=9 and synthetic N=16 and N=25. Its target is 16,000 puzzles and 110,000 primary runs in repetition 1; repetitions 2/3 from the old schedule are outside this denominator. This is a target, not a claim that every job succeeded or every puzzle was solved. The primary 10-second cutoff from START includes model construction/loading and solving. Tool applicability by N is frozen before measurement. The three AMO encodings use the same Glucose3 on eligible intersections; CP-SAT/HiGHS are broader baselines, while commercial solvers are limited to permitted smaller sizes. Non-applicability at larger N is not failure or timeout. License/unavailable remain separate. Public

puzzles retain source, original label, and hash; synthetic puzzles retain root, family, clue mask, and witness relation. Actual clue counts may differ from target rates through rounding. Content deduplication proves neither uniformity, uniqueness, difficulty, nor distinct isomorphism classes. Five masks from one root are a cluster, not five independent observations. Session receipts pin profile, worker count, and timing policy. Later workers are not assigned retrospectively. Sequential and parallel sessions remain separate; different IDs do not support a speedup claim. Paired timing requires identical IDs and eligible profile/session, with matched counts and both statuses disclosed. Primaries are selected by manifest, never by fastest batch or repetition. A closed consistent SQLite snapshot is the analysis source. RUNNING/INTERRUPTED, logs/ETA, future schedule, and diagnostic retry are not primaries. Diagnostic reruns do not replace primary observations or change their denominators. Coverage counts every primary status; runtime summarizes checker-passed SAT only. TIMEOUT is censoring at cutoff, not UNSAT or exact runtime. Unknown timing remains null. If PAR-2 is used, define its formula/denominator separately from runtime. One repetition provides no three-run median or repeatability. Percentiles across puzzles within a stratum describe instance variation, not repetitions. Public N9 and synthetic N16/N25 use different sampling mechanisms, so size is not isolated causally. Public ratings are not calibrated. Synthetic generators have bias and shared-root structure; clue fraction is not monotonic in difficulty. One valid witness does not prove uniqueness. Conclusions apply only to receipt-documented intersections, profiles, versions, and machines.

9. Experimental setup

The main batch `main-20261016-01` completed 906 fresh runs with a 10-second cutoff. Dataset, source code, configuration, and dependencies are frozen; `run.json` and `execution-freeze.json` retain metadata/hashes reconciled with the final analysis audit. Results in Section 12 belong only to this main batch.

9.1. Machine and runtimes

Runs are configured sequentially on the same Windows x64 machine, one job at a time and one thread/worker per engine. Table 3 distinguishes bindings from native runtimes where evidence exists. The native Glucose3 version is not inferred from PySAT, nor SciPy's embedded HiGHS from highspy.

Table 3
Machine and versions in the frozen main-run metadata.

Component	Recorded value
OS	Windows 11, 10.0.26300, x64
CPU	AMD Ryzen 7 H 260 w/ Radeon 780M Graphics; 16 logical CPUs
RAM	33,618,251,776 bytes (≈ 31.31 GiB)
Python	3.13.5
SAT	python-sat 1.9.dev16; Glucose3 (no separate native-version API in the probe)
CP-SAT	OR-Tools 9.15.6755
HiGHS	highspy 1.15.1; native 1.15.1
Gurobi	gurobipy 13.0.3; native 13.0.3
CPLEX MIP	cplex 22.2.0.1; native 22.2.0.0
CP Optimizer	docplex 2.32.264; native 22.2.0.0
Sparse matrices	SciPy 1.18.1

9.2. Data and comparison subsets

S80 comprises 60 synthetic instances: four sizes $N = 4, 9, 16, 25$, three target clue densities of 30%, 50%, and 70%, and five instances per group; plus 20 public 9×9 instances. Actual clue counts and seeds are in the frozen manifest. Public instances come from one Sudoku Exchange hard.txt file at commit `d8c8ebaee0c08c412cfba96af1923dfa61c83317` [14], sampled with seed 20261016 in the saved sample order.

B16 is chosen before results: sort each synthetic size–density group by actual seed and then instance ID and take the first (12 instances), plus the first four public instances in the fixed sample order. C10 is the $4 \times 4/9 \times 9$ part of B16: three synthetic instances at each size and the same four public instances. Thus $C10 \subset B16 \subset S80$. SAT runs three encodings on S80; CP-SAT/HiGHS use B16; Gurobi/CPLEX MIP/CP Optimizer use C10. SAT is not rerun separately for subsets. The main configuration has three repetitions per instance–configuration pair: $720 + 96 + 90 = 906$ runs, with a schedule shuffled using seed 20261016 and saved before execution. The pilot executed 55 jobs at repetition one to check integration/coverage, not 906 runs or three main repetitions. Pilot SAT/timeout rates and timings are not main Results.

9.3. Timing and process safeguards

Worker imports/input reading precede START. START/END use synchronized shared state and a monotonic clock. From START, the 10-second cutoff includes model/CNF construction, solver creation/loading, and solving; solving uses only the remaining budget. On-time END switches to a separate postprocessing watchdog. Decoding, validation, cleanup, and CSV/output I/O are outside build+solve. Missing or late END causes TIMEOUT; unknown timing components remain missing.

The Windows runner assigns the worker to a kill-on-close Job Object before releasing execution, terminates the process tree on deadline, and waits for descendants. The startup watchdog is 60 seconds before START; postprocessing has 30 seconds after on-time END. Startup/postprocessing/cleanup failure is ERROR, distinct from measurement timeout. Watchdogs do not extend solve time. The hard guard and measurement-end hook passed the pilot before use in the 906 fresh main runs.

CP Optimizer uses DOcplex 2.32.264 private preload to charge compile/send/load to build. Native TimeLimit is embedded in CPO before loading; after loading, cooperative abort_search is backed by the external guard. Exact post-load native TimeLimit changes are not promised; DOcplex upgrades require rechecking the private hook as in Section 7.

9.4. Aggregation rules

For each instance–configuration pair with three SAT runs, take the three-run median separately for build, solve, and total, then take the median of per-instance medians within the specified stratum. Components are aggregated independently: per-run total equals build+solve, with setup included in build, but the two group medians need not sum to the group total median. Paired runtime comparisons use only IDs with three SAT runs on both sides and report matched counts; solved counts and timeouts cover the full selected scope. Per-run PAR-2 uses SAT total or $2 \times 10 = 20$ seconds for TIMEOUT; ERROR, INVALID_INPUT, UNAVAILABLE, and LICENSE_LIMIT are excluded. The fastest repetition is not selected, and missing timings are not filled with zero.

10. Data source design and composition

The provenance audit verified the frozen index, manifest, witness metadata, and pinned source; its receipt and counts are retained locally. This section describes dataset construction and inference limits, not solver performance.

10.1. Public source, selection, and provenance

Ten thousand N9 puzzles come from four Sudoku Exchange files: 2,500 each from easy, medium, hard, and diabolical at commit `d8c8ebaee0c08c412cfba96af1923dfa61c83317` [14]. Original labels/ratings, URI, file/line provenance, and content hashes are retained. Group names are source metadata, not calibrated difficulty levels for the backends in this report. The source uses QQWing generation and Sukaku Explainer ratings; "public" therefore does not mean human-authored or naturally sampled. Deduplication compares canonical puzzle content rather than serialization strings. The audit records 20 duplicate/old candidates excluded and no exact duplicate inputs remaining in the frozen set; inputs do not overlap S80 or pilot. Public duplicate ownership follows easy, medium, hard, diabolical order and then ascending source line, avoiding dependence on randomized hash order. Hashes establish identity under the chosen canonicalization, not non-isomorphism.

10.2. Synthetic roots and legal transformations

The synthetic set has 5,000 N16 and 1,000 N25 puzzles, formed from 1,000 and 200 final roots, balanced between permutation and row_cycle within each size. Each root supplies five masks at requested clue levels of 20, 30, 40, 50, and 60 percent. All five levels from one root remain in one batch so their dependence is visible. A root is the completed grid after transformation, not an empty template or shared initial grid. The permutation family applies legal value relabeling, row reordering within bands, band reordering, and the analogous column/stack transformations. The row_cycle family starts from a permuted grid, selects two distinct rows in one band, and applies one complete nontrivial proper cycle to their value mapping across all columns. Cycle length is 2 through N-1; swapping the complete rows does not qualify. The audit records 600 such roots: N16 cycles of length 2 or 4 and N25 cycles of length 5. Counts describe transformations, not isomorphism classes. A mask retains $\lfloor pN^2/100 + 0.5 \rfloor$ clues. Integer counts give actual N16 levels 51, 77, 102, 128, 154 and N25 levels 125, 188, 250, 313, 375. Actual density may differ from the requested level. Masks are ranked independently by percentage and are not guaranteed to nest. The five masks share a root solution and transformation, so uncertainty analysis must cluster by root.

10.3. Witnesses, uniqueness, and deduplication

Frozen metadata contain 16,000 witnesses independently checked for domain, rows, columns, blocks, and clue preservation: 6,000 synthetic generation witnesses and 10,000 public witnesses obtained through the production solver under a separate 30-second acquisition budget. Acquisition is outside the 10-second benchmark cutoff and is not a benchmark result. Witnesses remain separate from solver inputs to prevent solution leakage. The provenance audit performed no new uniqueness solving. Public puzzles carry `source_claimed` only when a cited source makes that claim; synthetic uniqueness is unverified. One witness proves at least one solution exists, not that no others exist. Comparing public N9 with synthetic N16/N25 changes size, source, and generator together, so any performance difference cannot be attributed to N alone. Clue count is not a monotonic difficulty proxy.

11. Three-repetition baseline results

The analysis uses only the main batch `main-20261016-01`: 906 runs, 903 SAT outcomes accepted by the independent checker, and 3 TIMEOUT outcomes; there are no UNSAT, ERROR, INVALID_INPUT, LICENSE_LIMIT, or UNAVAILABLE outcomes. Pilot data are excluded. Table 5 reports both runs and instances with three SAT outcomes within each scope.

Table 4
Frozen composition from the verified index.

Group	N=9	N=16	N=25	Total
Sudoku Exchange	10,000	–	–	10,000
Permutation	–	2,500	500	3,000
Row cycle	–	2,500	500	3,000
Synthetic total	–	5,000	1,000	6,000
Total	10,000	5,000	1,000	16,000

Table 5
Coverage of the 906 main runs.

Configuration	Scope	SAT/runs	Instances with 3 SAT	TIMEOUT
Glucose3 pairwise	S80	240/240	80/80	0
Glucose3 sequential	S80	240/240	80/80	0
Glucose3 bitwise	S80	240/240	80/80	0
CP-SAT	B16	48/48	16/16	0
HiGHS ILP	B16	45/48	15/16	3
Gurobi MIP	C10	30/30	10/10	0
CPLEX MIP	C10	30/30	10/10	0
CP Optimizer	C10	30/30	10/10	0

11.1. CNF size and construction cost

Figure 2 fixes clue density at 50% to isolate grid-size effects; vertical axes are logarithmic, variable counts include auxiliaries, and counts precede preprocessing. Seed 0 is representative because counts agree across all five seeds within each size–density stratum. Table 6 separately illustrates N25/30% with 188 clues: pairwise has no auxiliaries but the most clauses; sequential has more auxiliaries than bitwise but fewer clauses. CNF counts are not proxies for native CP/MIP model sizes.

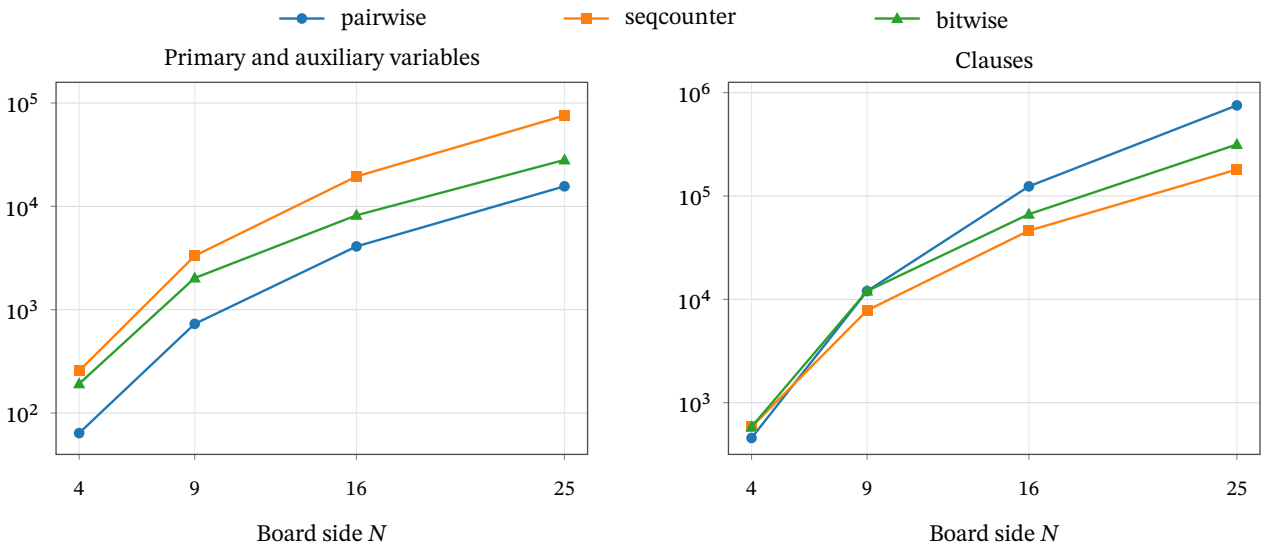


Figure 2: Synthetic CNF size at fixed 50% clue density; seed 0 at each size, before preprocessing.

Table 6
Measured CNF for synthetic-N25-p30-s0, 188 clues; before preprocessing.

Encoding	Primary	Auxiliary	Total variables	Clauses
pairwise	15625	0	15625	752688
sequential	15625	60000	75625	180188
bitwise	15625	12500	28125	315188

Table 7 uses the same five IDs, `synthetic-N25-p30-s0` through `s4`, each with three SAT runs for all encodings. Build, including solver creation/loading, exceeds solve time in this stratum. Sequential has the lowest median total even though bitwise has a lower median solve time. Thus solve-only timing or variable counts omit a substantial pipeline cost. Figure 3

Table 7
Synthetic 25 × 25, 30% clues: five common IDs, seconds.

Encoding	Median build	Median solve	Median total
pairwise	0.381882	0.022380	0.402449
sequential	0.130810	0.024383	0.159527
bitwise	0.193860	0.015416	0.210650

Each column is an independently aggregated median of per-instance three-run medians; the group build and solve medians do not sum to the group total median.

separates the three densities; each point is the median of five per-instance medians, each based on three repetitions. Rankings are not fixed: for synthetic 4 × 4/30%, pairwise/sequential/bitwise median totals are 0.001189/0.001520/0.001582 seconds. Sequential's lowest total at N25/30% does not imply the lowest time for every n or dataset.

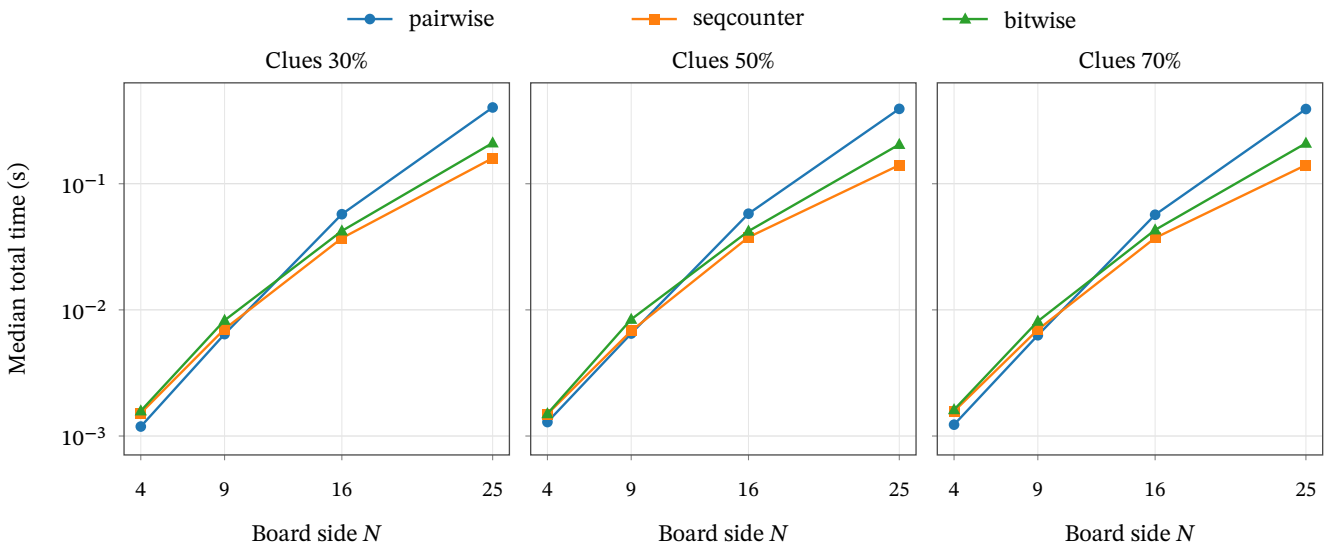


Figure 3: SAT median total (seconds), five common synthetic IDs per point; three repetitions/ID, with a 10-second cutoff for build (including setup) + solve. No SAT encoding times out.

11.2. Baselines on matching instance sets

Tables 8–10 come from `paper/baseline-matched.csv`, using pairwise as a fixed reference rather than choosing the fastest encoding separately for each opponent. Median totals include only IDs with three SAT runs in both configurations. In B16 each density at N25 has one ID, unlike the five-ID S80 statistic in Table 7. For example, on `synthetic-N25-p30-s0`, CP-SAT takes 0.230552 seconds versus pairwise's 0.394866 seconds; the latter must not be replaced by the five-ID median of 0.402449 seconds. On the same four public instances, CPLEX MIP has a median total of 0.004748 seconds, below pairwise's

Table 8
B16 synthetic: median total on each common ID, seconds.

N	Clues (%)	m_{CP}/m_H	CP-SAT	HiGHS	Pairwise
4	30	1/1	0.004866	0.001704	0.001184
4	50	1/1	0.004223	0.001638	0.001453
4	70	1/1	0.004099	0.001468	0.001323
9	30	1/1	0.013557	0.012148	0.006350
9	50	1/1	0.006484	0.004400	0.006544
9	70	1/1	0.005491	0.004467	0.006700
16	30	1/1	0.057879	0.047280	0.057086
16	50	1/1	0.009205	0.021862	0.058682
16	70	1/1	0.007168	0.022792	0.055755
25	30	1/0	0.230552	–	0.394866
25	50	1/1	0.055806	0.113672	0.382179
25	70	1/1	0.012724	0.066873	0.383525

Here m counts IDs with three SAT runs in both configurations. Each row has one ID, unlike five IDs per S80 stratum. A dash denotes three HiGHS timeouts at N25/30%, with no SAT median.

Table 9
C10 synthetic: one common ID per row, median total (seconds).

N	Clues (%)	m	Gurobi	CPLEX MIP	CP Optimizer	Pairwise
4	30	1	0.002521	0.002137	0.029427	0.001184
4	50	1	0.002477	0.002280	0.033032	0.001453
4	70	1	0.002210	0.002362	0.030086	0.001323
9	30	1	0.010647	0.006728	0.029006	0.006350
9	50	1	0.007375	0.003787	0.029938	0.006544
9	70	1	0.007339	0.003884	0.030118	0.006700

Table 10
Four common public 9×9 instances in B16/C10, median total (seconds).

Backend	Scope	m	Median total (s)	Pairwise
CP-SAT	B16	4	0.006506	0.006534
HiGHS ILP	B16	4	0.004704	0.006534
Gurobi MIP	C10	4	0.008019	0.006534
CPLEX MIP	C10	4	0.004748	0.006534
CP Optimizer	C10	4	0.030911	0.006534

The same four IDs are L069940, L293964, L269904, and L215069 (prefix public-se-hard-). Pairwise is a fixed reference, not the fastest encoding selected separately for each opponent.

0.006534 seconds; Gurobi at 0.008019 and CP Optimizer at 0.030911 seconds are higher than pairwise. This counters an assertion that SAT always wins; it does not establish that ILP or one solver is always best. Ratios are not reported in these tables; a paired ratio would require the median of per-ID ratios rather than division of group medians. Millisecond differences on a small sample require caution.

11.3. Timeouts, PAR-2, and traceability

All three timeouts are the three HiGHS repetitions of `synthetic-N25-p30-s0`. This ID has no SAT median; the runtime intersection between HiGHS and SAT/CP-SAT on B16 contains 15 IDs, while its intersection with commercial engines on C10 still contains 10. Two SAT encodings share 80 IDs. Each timeout receives PAR-2 $= 2 \times 10 = 20$ seconds, for a total timeout penalty of 60 seconds; mean PAR-2 for the HiGHS N25/30% stratum is 20 seconds. Missing timings are not zero: r3 lacks build/solve/total and remains missing, while r1/r2 have observed totals near the cutoff but remain TIMEOUT. Successful-only runtime statistics must be read with the counts in Table 5. Tables and figures use the locked final analysis; `summary.csv`, `instances.csv`, `matched-pairs.csv`, and `cnf.csv` retain detailed statistics. `paper/raw-row-index.csv` maps job IDs to original CSV rows; the audit retains configuration, versions, and hashes. Pilot data are not merged, and the benchmark is not rerun to select favorable results.

12. Expanded results, repetition 1

This analysis covers all 16,000 selected puzzles and 110,000 primary jobs. It contains 109,664 checker-passed SAT outcomes and 336 non-completions. NC is a post-hoc display grouping of TIMEOUT and ERROR; raw statuses remain unchanged.[1] Every puzzle has at least one SAT outcome across methods, which does not mean every solver solved every puzzle. Table codes are PW pairwise, SC sequential counter, BW bitwise, CP CP-SAT, HG HiGHS, GR Gurobi, CX CPLEX MIP, and CO CP Optimizer. Table 11 gives configuration denominators. By size, non-completions are 0/80,000 at N9, 2/25,000 at N16, and 334/5,000 at N25. The coverage table retains applicable denominators by method. Table 12 uses puzzles rather than jobs as its unit: every puzzle has at least one primary SAT. It does not establish that all methods cover the same puzzles.

12.1. Runtime on common-SAT intersections

The runtime summaries condition on SAT and retain the stated instance intersections. The main panel selects the largest session by coverage, not speed: N25/permutation/40% clues, 32 workers, resource-contended profile. ECDF, quantiles, and build/solve components use the same 86 common SAT IDs across three encodings; these are single measurements per puzzle, not medians across three repetitions. PW has a median of 0.741841 s, SC 0.411756 s, and BW 0.432963 s; corresponding p95 values are 1.353671, 1.072038, and 0.897043 s. These percentiles are across puzzles in one run, not repeatability estimates. No confidence intervals or isolated speedups are estimated; the session is resource-contended and synthetic instances share roots. The scatter uses its separately declared PW/SC paired set in figure-scope; no causal interpretation is made across sessions or configurations. The 24/40-worker sessions support coverage/timeline only and are excluded from paired performance claims. Clue fraction is not treated as a linear proxy for difficulty. Figure 9 conditions on SAT, separates families, and carries sample counts in the source data; masks from one root are dependent.

[1] The audit records 335 TIMEOUT outcomes and one premeasurement ERROR within this group; identity and evidence are retained in the appendix and hash-pinned artifacts.

Table 11
Primary coverage by N and method; NC is the post-hoc display group TIMEOUT+ERROR.

N	Method	Jobs	SAT	NC
9	CX	10000	10000	0
9	CO	10000	10000	0
9	CP	10000	10000	0
9	GR	10000	10000	0
9	HG	10000	10000	0
9	BW	10000	10000	0
9	PW	10000	10000	0
9	SC	10000	10000	0
16	CP	5000	5000	0
16	HG	5000	4999	1
16	BW	5000	5000	0
16	PW	5000	5000	0
16	SC	5000	4999	1
25	CP	1000	1000	0
25	HG	1000	666	334
25	BW	1000	1000	0
25	PW	1000	1000	0
25	SC	1000	1000	0

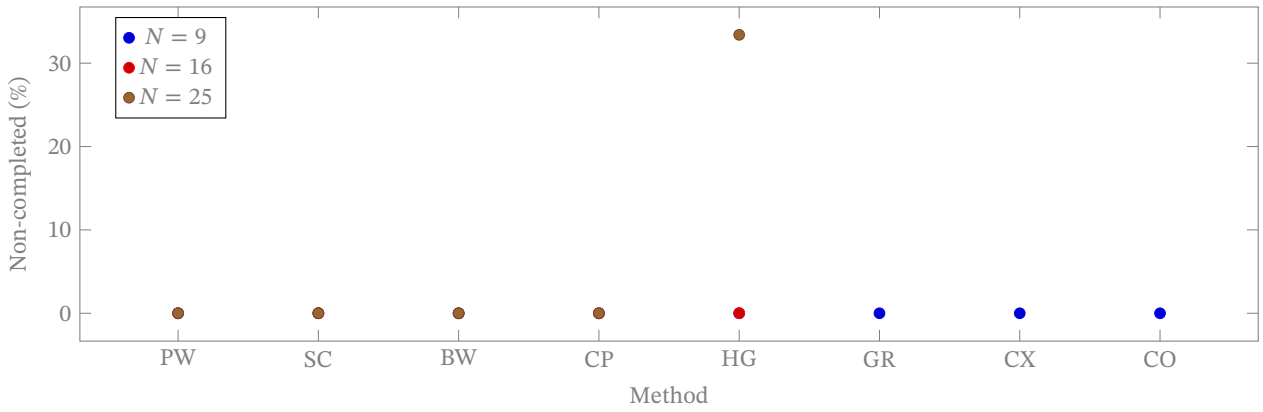


Figure 4: Non-completion rate by size and method; denominators are applicable jobs, not a measure of difficulty.

Table 12
Puzzles with at least one primary SAT by N.

N	Attempted	AnySAT	NoSAT
9	10000	10000	0
16	5000	5000	0
25	1000	1000	0

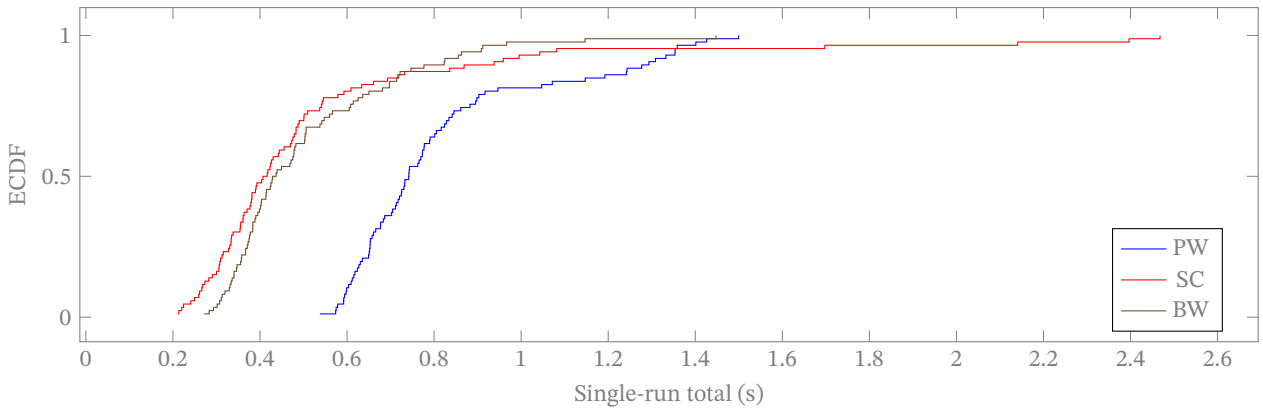


Figure 5: Total-runtime ECDF on 86 common-SAT puzzles; contended session, one repetition.

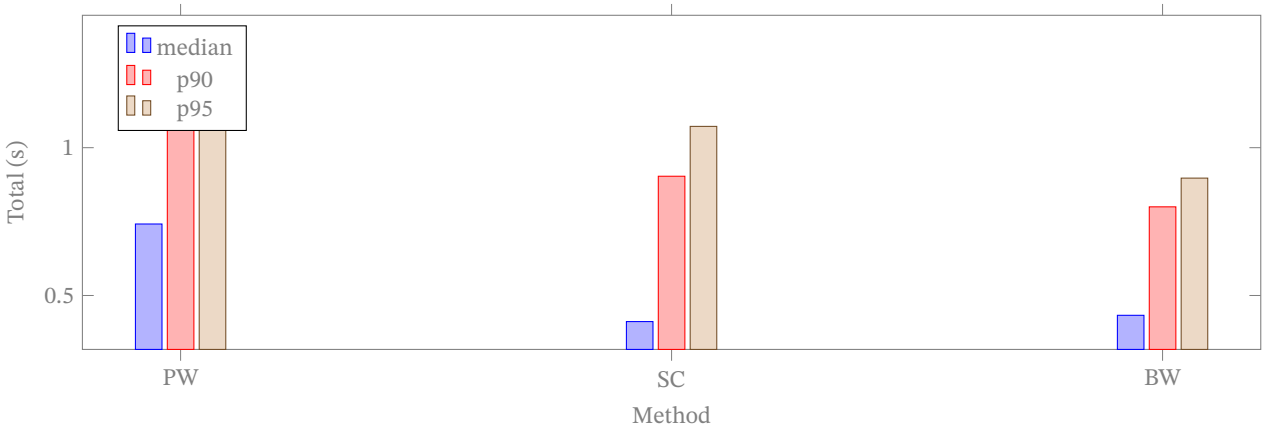


Figure 6: Median, p90, and p95 across 86 common-SAT puzzles; percentiles describe between-instance variation.

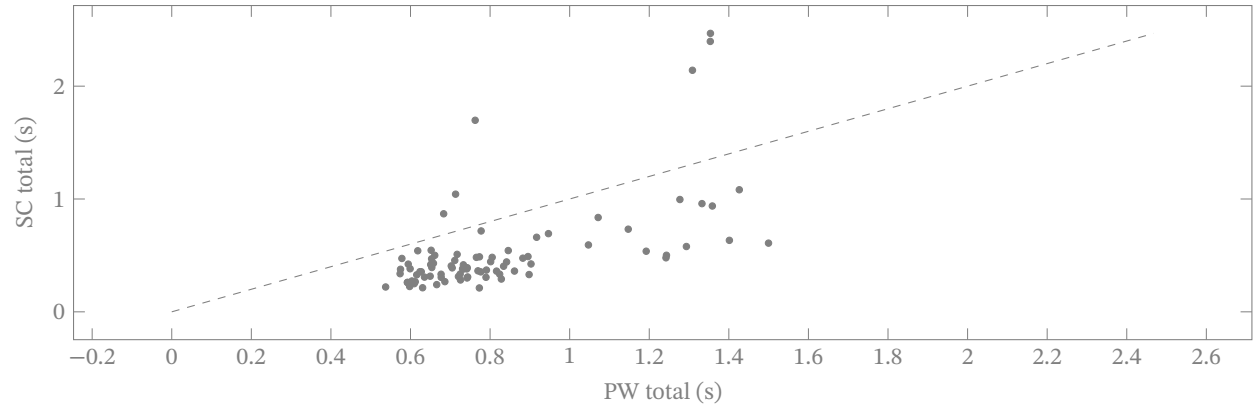


Figure 7: Paired PW–SC runtime on the same 86 puzzles; diagonal indicates equal runtime.

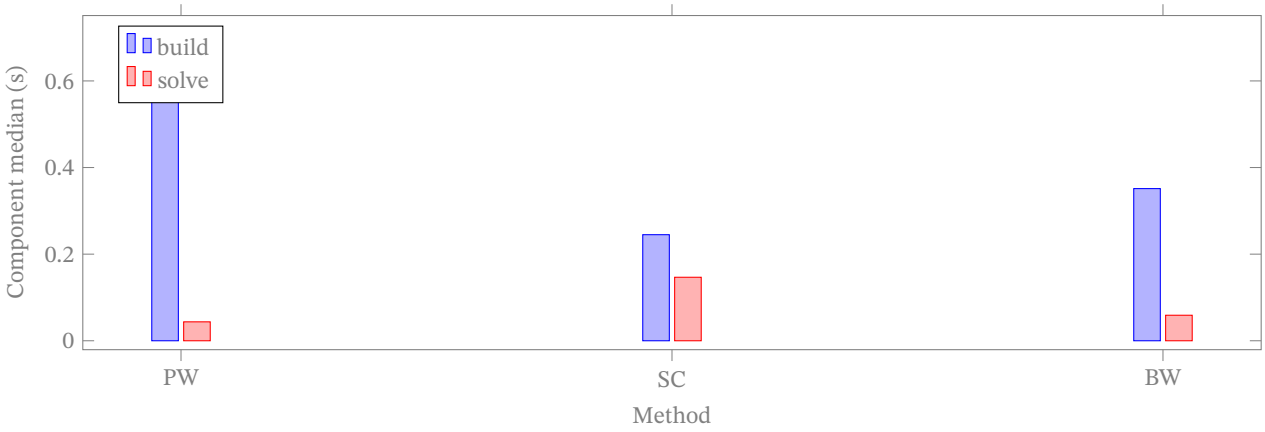


Figure 8: Median build/setup and solve components on the same 86 puzzles; component medians do not sum to the median total.

12.2. CNF size and profile coverage

Variable counts include auxiliaries; clause counts exclude clue units and precede preprocessing. Points use the representative configurations specified in figure scope and do not stand in for native CP/MIP model size. The profile timeline shows coverage by session start time, not execution rate. The 24/40-worker trials remain for operational coverage and do not support performance conclusions. One repetition does not support three-run medians or estimates of repeatability across repetitions. Historical repetition-2 and repetition-3 rows (324 and 343) are excluded; diagnostic retries do not replace primary outcomes. 24/40-worker trials are excluded from paired runtime analysis. Detailed values and raw statuses remain in hash-pinned CSV artifacts; report tables retain denominators without listing every ID.

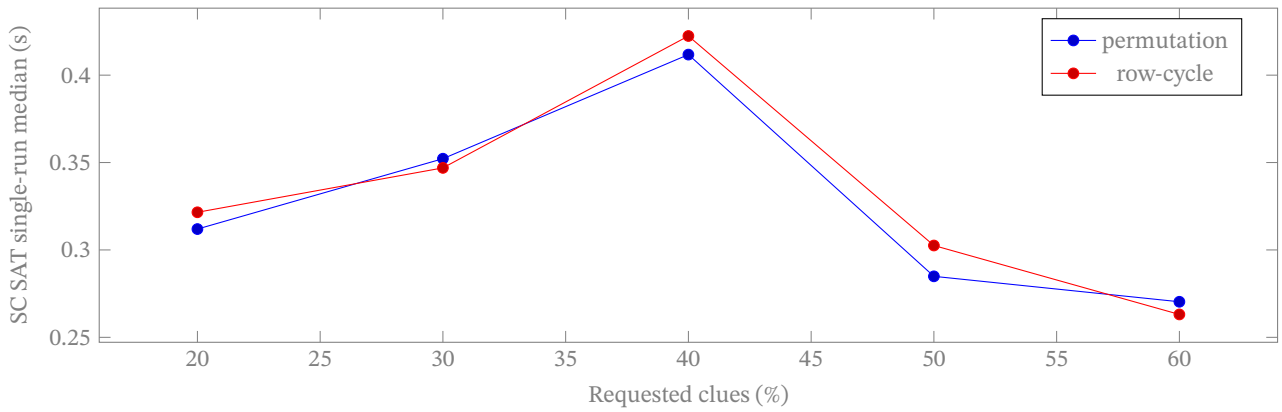


Figure 9: SC runtime by clue level in the N25 session; descriptive across puzzles, not a causal difficulty relationship.

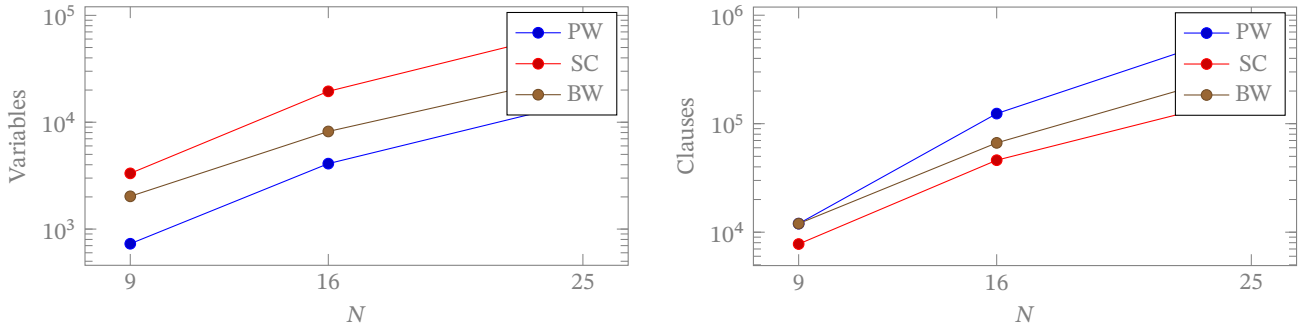


Figure 10: CNF variable and clause counts by grid size, before preprocessing.

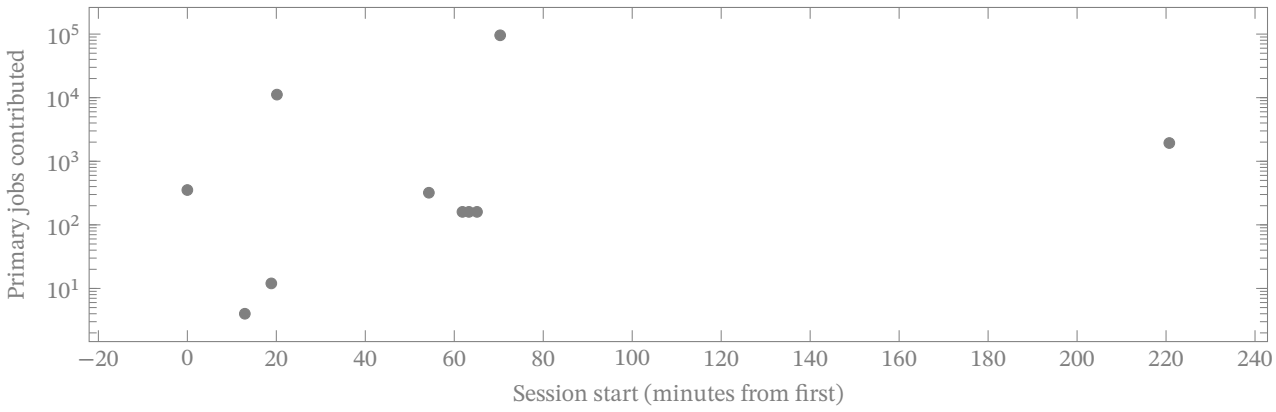


Figure 11: Primary jobs by session; relative time axis and logarithmic scale.

Table 13
Coverage by session, profile, and workers; 24/40-worker trials indicate coverage only.

Session	Workers	Jobs	SAT	NC
80292632	1	353	353	0
781a1ce5	4	4	4	0
ed7d42b0	4	12	12	0
f17d0075	4	11171	11136	35
4bd685e0	8	320	318	2
d5efc155	8	160	160	0
c6569a16	24	160	159	1
b4437fae	40	160	159	1
a23e8969	32	95724	95433	291
4e0782e3	32	1936	1930	6

13. Limitations

Synthetic instances originate from structured valid grids and transformations, introducing generator bias rather than representing all Sudoku. Larger grids or fewer clues do not establish greater difficulty. Public instances are also externally machine-generated, with only 20 sampled from one hard.txt bucket; bucket names/ratings are uncalibrated source labels. The 80 instances have SAT evidence but unverified uniqueness; finding one valid completion is not a uniqueness certificate.

Baselines are supplementary small samples: compare three SAT encodings on S80, CP-SAT/HiGHS on B16, and commercial engines on C10 or the available intersection. Paired runtimes use instances with three successful repetitions in both configurations and disclose scoped counts/coverage. Medians from different sets are not a common-sample comparison. A 10-second timeout is cutoff-censored, not true solving time. ERROR, UNAVAILABLE, and LICENSE_LIMIT are distinct from TIMEOUT and excluded from timeout PAR-2 penalties.

Free editions restrict coverage: Gurobi size-limited allows 2,000 variables/2,000 linear constraints; CPLEX MIP Community allows 1,000/1,000. The $4 \times 4/9 \times 9$ C10 models fit and completed all three main repetitions on C10; this does not imply all-S80 coverage. CP Optimizer has a model/search-space-dependent limit: a size failure on one 16×16 or 25×25 grid does not establish failure for every grid of that size. HiGHS is not relabeled as Gurobi/CPLEX.

One machine, one version configuration, three repetitions, and a finite cutoff do not establish generality across hardware/solvers. B16 has one ID per synthetic stratum; C10 contains only small grids and four common public instances. Millisecond differences are sensitive to overhead; no statistical significance testing, RAM measurement, or search counters are provided. Baselines use models and engines different from SAT, so differences cannot all be attributed to encoding. Smoke/pilot are separate integration gates and are excluded from main results.

The expanded analysis has one repetition: 109,664 SAT, 335 TIMEOUT, and one ERROR among 110,000 primary jobs. Historical repetition-2/3 rows (324/343) and diagnostic retries are excluded. NC=336 is a post-hoc display grouping; raw TIMEOUT and ERROR statuses remain separate. The WinError 6 guard error preceded measurement and has null timings; it is not a cleanup failure. The N25/permutation/40% panel is one resource-contended 32-worker session and does not estimate isolated speed. The ECDF, quantile, and component panels use 86 common SAT puzzles; percentiles describe between-instance variation in one measurement, not repeatability. The pairwise scatter uses its separately declared matched sample. No confidence interval or uncertainty estimate is provided; masks from one root are correlated. The 24/40-worker trials contribute coverage only. N9 public and N16/N25 synthetic coverage changes source, generator, and size together, so it cannot identify a causal size effect.

14. Conclusion

The cell–value Boolean model compares pairwise, sequential counter, and bitwise encodings under one Sudoku specification and Glucose3 solver. The three-repetition baseline on 80 puzzles and the expanded one-repetition analysis answer questions on separate denominators. In the expanded N25/permutation/40% panel, the 86 common SAT puzzles have PW/SC/BW median runtimes of 0.741841/0.411756/0.432963 seconds. This is one resource-contended session, not evidence that SC always wins. Build/setup remains a distinct runtime component and must be retained in interpretation. The expanded run completes the selected identity/coverage scope with 110,000 primary jobs: 109,664 SAT and 336 non-completions. Having at least one SAT across methods for all 16,000 puzzles does not mean every method solves every puzzle. This report does not estimate repeatability across repetitions or isolated speedup. Future work should add repetitions and machines, test uniqueness, preserve root clusters, and evaluate uncontended sessions.

Code and data availability

The public repository is `https://github.com/Ngynkybyrd/2627I_INT3011E`; the baseline was published at snapshot `f45d378`. The expanded report, snapshot, analysis, and PDFs for revision `final-20261020` are retained locally; the expanded artifacts are not public or pushed. The source manifest, analysis receipt, and PDF hashes are recorded in this revision's handoff. Section 12 reports repetition 1 of the expanded analysis; Section 11 preserves the separate three-repetition baseline.

A. Job lifecycle and process protection

Imports and input reading precede START. A monotonic clock measures build/CNF, solver initialization/loading, and solve within the remaining budget; decoding, checking, and cleanup use a separate postprocessing watchdog. Late results are not admitted as SAT. Unobserved timing after cutoff remains null, never filled with ten seconds. The runner uses a Windows kill-on-close Job Object and waits for the process tree; watchdog time does not extend solve. Exception handling stops solving, cancels/joins timers and callbacks, confirms worker exit, and then deletes native objects. Join/cleanup errors are explicit, not hidden by solver status. Recovery claims only jobs without a valid primary terminal; ownership/fencing prevents concurrent writers and stale publication. Result, witness/hash, attempt, and terminal status need atomic publication or explicit reconciliation when artifacts are outside the database. Statuses are defined observations: INVALID_INPUT stops at parsing; SAT requires checker-pass; UNSAT requires infeasibility evidence; TIMEOUT is cutoff before an admissible result; ERROR retains worker/guard/decode/checker/adapter failures. UNAVAILABLE and LICENSE_LIMIT are separate.

A diagnostic retry does not replace primary ERROR. Separate selected-scope-complete from error-free because all identities may be present while a historical ERROR remains.

B. Aggregation and reproducibility contract

Coverage is the complete denominator by configuration/stratum; runtime is conditional on SAT. Paired comparisons require the same ID and eligible profile/session, with counts for both sides. Do not divide medians from different samples. With timeouts, show censoring and success coverage alongside runtime; do not combine ERROR/invalid/license/missing with timeout. Percentiles across instances are not repeatability. Do not treat dependent masks/root as independent for uncertainty. The result receipt pins snapshot hash, schema/scope, every artifact, and analysis outputs. The reader checks SQLite integrity, expected-versus-observed matrix, unique identities/configurations, raw status, timing nullability, SAT solution/hash/checker, and session metadata. Tables are generated from checked summaries, not edited numerically by hand. The paper consumer verifies hashes; captions and prose share denominators. Final analysis checklist: identity/coverage; SAT/TIMEOUT/ERROR status; composition by N/source/family/root/clue; theoretical-versus-observed encoding counts; successful runtime percentiles; paired IDs; timeout denominator; profile timeline; ERROR case; limitations. Every summary row traces to CSV, then snapshot and receipt. Missing fields or unexplained status stop publication rather than justify dropping rows. When integrating final figures, the abstract, captions, and conclusion use the same denominator/hash. If

Table 14
Artifacts required before integrating results.

Artifact	Control
SQLite snapshot	Consistent backup, integrity/FK, read-only hash
Receipt	Scope/schema, quality flag, artifact hashes
Run/session CSV	Identity, raw status, timings, profile provenance
Analysis summary	Denominators, censoring, matched counts, lineage
Paper source/PDF	Tables/figures generated from analysis, font/layout QA

the manifest records an infrastructure error, report its quality flag alongside coverage. Local staging is not a public URL.

B.1. Non-completed primary status audit

The NC group in the results is a post-hoc display group comprising exactly 335 TIMEOUT outcomes and one ERROR, for 336 total. TIMEOUT is censored at the 10-second cutoff and is not assigned an imputed runtime. The single ERROR is a premeasurement guard WinError 6 with null build/solve/total timings; it is not UNSAT, timeout, or cleanup failure. A diagnostic retry does not replace the primary. The coverage table and `declared-error.csv` retain the counts and audit evidence; raw statuses remain distinct.

C. Execution lifecycle, ownership, and recovery

A job state machine distinguishes PENDING, RUNNING, and terminal states. The RUNNING claim commits before a worker calls a solver. Result, witness/check evidence, and terminal attempt commit together after execution; external data require temp-write, fsync, atomic rename, and receipt-linked reconciliation. Only one owner solves a job. An expired lease or missing PID does not prove an old worker stopped; recovery verifies process-tree exit and ownership fencing against late writes. If a process dies after solving but before commit, an observation may be lost; the protocol does not claim exactly-once execution. Resume does not rerun terminal SAT/TIMEOUT/ERROR rows. Only PENDING or orphan INTERRUPTED work may be claimed after proving the previous owner/worker exited. All attempts remain retained. A bad candidate, mismatched witness, cleanup failure, or out-of-taxonomy status preserves evidence and halts for adjudication rather than skipping forward. A final snapshot comes from a consistent backup, after which readers open only the snapshot. CSVs are reproducible projections, not a new source of truth. The receipt links manifest, schedule, raw database snapshot, solution/witness storage, CSVs, analysis summary, tables, and figures by content hashes. Regenerated layers identify parent hashes and tool version; CSVs are never edited by hand or silently rehashed over old outputs. The diagram separates the measurement boundary



Figure 12: Measurement path: checking and commit fall outside solver cutoff but remain in the accepted lifecycle.

from publication conditions. Build/solve produces raw status and timing; decoding/checking decides whether a solution is admissible; commit records one primary with witness and attempt. A failure at one stage is not relabeled based on a later stage. For example, a diagnostic SAT retry does not turn a primary ERROR into SAT.

Table 15
Identity and provenance fields used in analysis.

`instance_id`	Frozen canonical puzzle, stable across configurations
`configuration`	Specific backend and encoding
`batch_id`	Puzzle/job position in frozen schedule
`session_id`	Execution unit with its own receipt
`profile, workers`	Actual session profile and worker count
`timing_policy`	Measured components, boundary, and cutoff
`repetition`	Primary selector; rep1 for expanded-report scope
`raw_status`	Direct result before taxonomy/checker admission
`build/solve/total`	Observed times; unknown values remain null

D. Field and status dictionary

Statuses follow observed events: malformed input is INVALID_INPUT; a valid on-time candidate becomes SAT after checking; infeasibility evidence yields UNSAT; measurement cutoff before an admissible result yields TIMEOUT; guard, worker, decode, checker, or adapter failure yields ERROR; unavailable runtime or license limits remain distinct. Timeout does not establish infeasibility. Keep cleanup status separate where the schema permits. Coverage should distinguish target, matrix-planned, selected primaries, observed primaries, SAT, TIMEOUT, ERROR, and excluded statuses. Each denominator answers a different question. A complete selection can contain an ERROR and is therefore not synonymous with error-free. Runtime medians report finite SAT counts; matched ratios report pairs after filtering; a zero denominator is null, not infinity.

E. Tracing checklist from paper to data

1. Match snapshot hash/schema to receipt; verify SQLite integrity/FKs.
2. Reconcile expected identities/configurations/repetitions to all primaries; retain ERROR/TIMEOUT.
3. Validate witness/hash/checker for every SAT; keep diagnostic retries outside primaries.
4. Take session/profile/worker/timing policy from receipts; do not assign retrospectively.
5. Generate coverage, composition, paired/runtime, and timeout tables from analysis; captions disclose denominators.
6. Check abstract/prose/conclusion figures against artifact hashes; separate baseline and extension claims.

F. Metric definitions and denominators

Within a defined stratum, coverage classifies every primary by observed status. Completion rate is admissible primaries over applicable primaries in the same sample; report numerator and denominator, not only a percentage. A configuration excluded by the matrix is outside the denominator. An applicable job ending ERROR, TIMEOUT, or without valid terminal evidence remains in coverage. Runtime analysis conditions on checker-passed SAT primaries. The success set $S_{c,g}$ for configuration c and group g contains IDs with finite admissible timing. Its empirical CDF is $F_{c,g}(t) = |\{i \in S_{c,g} : T_i \le t\}|/|S_{c,g}|$. Always pair $|S|$ with the group's full primary count; an ECDF cannot replace completion rate. With one observation per ID, median/p90/p95 describe puzzle variation. For type-7 quantiles, with m ordered observations and quantile q, the zero-based index is $h = (m-1)q$, interpolated between floor(h) and ceil(h). For m=0 the statistic is undefined; for m=1 it equals that observation. Do not imply extra precision by printing more digits than timer resolution supports. An A/B matched sample is the intersection of eligible IDs within the same N/source/family/clue/profile/session rule. Per-ID ratio is $R_i = T_{A,i}/T_{B,i}$; report median R_i, count, and status filters. A ratio below one means A is shorter on that matched sample. Do not divide group medians computed on different IDs. TIMEOUT lacks a fully observed runtime. A censored-runtime plot may show cutoff markers only if execution records contain event times; with terminal status alone, coverage plots are appropriate. ERROR is not censored runtime because it has a different cause. If analysis uses PAR-2, disclose its formula, penalty, denominator, and that it is not measured latency.

G. Threats to validity

G.1. Construct validity

The task is finding one valid completion, not assessing uniqueness, aesthetics, or human difficulty. Models reduce it to feasibility. The three AMO representations differ in clauses while sharing primary variables, ALO, EXO groups, and clue order; runtime differences include propagation and construction/loading cost. Clause count is not a direct measure of difficulty or algorithm quality. The ten-second cutoff is an operational rule, not the exact solve time of a timed-out job. Build+solve total includes encoding cost but depends on API/runtime versions. It cannot be directly compared with native-solve-only studies or different cutoffs without measurement normalization.

G.2. Internal validity

Worker scheduling, CPU frequency, thermal limits, memory pressure, cache warmth, and I/O may affect times. Session receipts identify profiles and overlap but do not remove confounding. Four-worker builds may contend for resources; cross-session differences compare environmental/profile bundles, not pure thread-count or algorithm effects. Deadline events and measurement boundaries must use the same implementation version. A guard error before START differs from timeout after START. Late workers, cleanup failure, missing native timing, and stale ownership may cause missing profile observations or ERROR. Primary/error lineage preserves evidence while retries remain diagnostic. Only a QA-passed receipt ensures downstream analysis does not hide these distinctions.

G.3. External and statistical validity

One machine and software stack limit transfer to other hardware/backends. Public N9 and synthetic N16/N25 use different generators/sources; roots induce dependence, and the number of roots is smaller than the mask count. One repetition does not estimate run-to-run noise. Thus a raw 16,000 sample size is not equivalent to 16,000 independent puzzles, much less representative of all Sudoku. Family/root analyses must preserve grouping across strata. Public ratings describe source groups only. Do not infer causality from clue-percentage regression when generator, N, and family covary. Future studies can increase machines/repetitions, independently test uniqueness, diversify base roots, pre-register comparison strata, and separate build/solve under aligned metrics. These are future directions, not current results.

H. Correctness and fault-injection matrix

Table 16
Core contract cases and expected classification.

Condition	Class	Evidence retained
Bad shape/domain	INVALID_INPUT	No solver construction
Valid conflicting clues	UNSAT or status	Raw solver evidence
Valid completion	SAT	Decode + checker receipt
Row-invalid candidate	ERROR	Checker rejection
Cutoff before result	TIMEOUT	Primary attempt; unknown times null
Pre-START guard failure	ERROR	Raw error, no fake timing
Diagnostic retry SAT	Primary unchanged	Parent link + separate receipt

Core model tests are separate from benchmark data. On small N, enumerate completions and compare CNF/CP/ILP feasibility; at N=4, compare the CNF projection onto primary variables with an independent Sudoku enumerator. This checks encoding without running the benchmark. Mapping tests cover boundaries at N=4/9/16/25; auxiliary-collision tests reserve top IDs; metamorphic tests relabel values/bands/stacks and check validator invariance. Fault tests inject timer-join failure, delayed callbacks, nonzero worker exit, descendants after timeout, database crash before/after commit, corrupted witness hashes, and stale fencing tokens. Expected behavior is explicit status, no late publication, and a recoverable receipt. Assert negative properties: no duplicate simultaneous solver, no terminal rerun, no SAT after checker rejection, no cleanup while a timer is active, and no frozen-manifest mutation. Analysis tests separately cover denominators, missingness, censoring, profile mismatch, and root dependence. Fixtures test analyzer logic; real-data acceptance requires snapshot hash/schema, all IDs/jobs, witnesses, and solution checks. Fixtures or pilot do not fill observed coverage.

I. Questions and caveats for result figures

Table 17
Questions and limits for the figure families.

Figure	Question	Caveat
Completion by N/config	How do statuses vary?	Not runtime ranking
SAT runtime ECDF	How does SAT tail vary?	Conditional on success
Clue/family strata	Coverage by composition?	Roots are dependent
CNF size	Clause/variable tradeoff?	Does not predict runtime
Build vs solve	Where is time spent?	Accepted times only
Matched comparisons	What happens on same IDs?	Count and profile
Session timeline	How do profiles/overlap differ?	Operational description

Each chart links a prespecified question to the relevant measurement and denominator. Coverage bars use all primaries; runtime ECDFs use SAT-only and show solved fraction; build/solve scatterplots require both finite components; matched figures label n; timelines separate sessions; composition plots describe sample/root structure. Never plot expected values or interpolate missing strata.

J. Walkthrough from input to admissible result

An input passes through the following stages in order; a later stage cannot rewrite the meaning of an earlier result.

1. Read exact bytes, verify source/content hashes, decode schema, and validate shape/domain. Invalid input ends as INVALID_INPUT before timing.
2. Normalize clue triples while retaining links to instance ID, N, source, and root. Do not infer solutions or remove conflicting clues.
3. Build with the pinned builder; record versions/configuration, top variable ID, clause/row counts, and START/END per timing contract.
4. The adapter calls the configured single-thread solver under remaining budget, native cutoff/interrupt, and hard worker guard. Preserve the raw result before classification.
5. If SAT, the decoder reads primary variables only, creates a completion candidate, then an independent checker verifies domain, clues, and every unit.
6. Cancel and join timers/callbacks; verify worker-tree cleanup. Admit SAT only if solve finishes by deadline, checker passes, and cleanup/status satisfy policy.
7. Commit primary result, status, timing, witness/hash, and attempt under contract. Later analysis reconciles the job matrix and session receipt from the snapshot.
8. The consumer generates tables/figures from checked analysis, pins output hashes, and validates figures in the PDF. Any reconciliation failure blocks integration.

This sequence separates three evidence classes: structurally valid model, raw solver status, and independently accepted candidate. Only the third supports an SAT claim. Solver status matters for UNSAT/TIMEOUT but does not replace candidate checking. The paper should not shorten this to “solver found SAT” when the pipeline requires a checker gate.

K. Table generation and reconciliation

Every results table has a clear machine-readable source. Coverage and composition count from primary snapshot, not whichever CSV files happen to be present. Runtime aggregates use accepted observations; paired tables use intersected IDs and status filters; profile tables join session receipts; encoding counts derive from builder diagnostics or stated formulas. Captions identify units and denominators. Independent reconciliations include: status rows sum to observed primaries; per-configuration coverage matches applicability; runtime m may be below SAT coverage when timing is missing but never above it; paired n does not exceed the filtered ID intersection; source/family/N composition sums to unique instances; figure points reconcile to summary rows. Quantile definitions must agree across analysis JSON, LaTeX tables, and prose. For small groups, keep counts adjacent to estimates rather than in a distant appendix. With zero successful timings, median is undefined and remains null/N/A, never zero. Compact tables may round for readability while CSV retains precision and documents the rounding rule. When historical ERROR remains in primaries, selection may contain every identity while not being error-free. Headline tables may group non-completed statuses under an agreed reporting rule, while the audit appendix retains raw categories and receipts. Presentation grouping does not change raw status, denominators, or quality flags. Diagnostic retries retain separate lineage. PDF text extraction helps find missing figures, unresolved references, table labels, and metadata but cannot prove layout. Render-check pages with long tables, plot axis labels, footers/footnotes, displayed math, and page transitions. Every QA record names the build ID so reviewers do not confuse new source with an old PDF.

L. Comparison scope and fairness

The SAT encoding comparison holds Glucose3, builder conventions, clue order, and model semantics constant; AMO clauses are the main change. Auxiliary counts, clause width, propagation order, and library data structures still differ and belong to the treatment. Findings attach to the specific PySAT/library version, not every pairwise/seqcounter/bitwise implementation. CP-SAT uses an AllDifferent model; binary ILP uses one-hot equalities and zero objective; MIP backends receive the same CSR matrix, bounds, and integrality. Internal representations differ, so the comparison concerns implemented execution pipelines, not an isolated constraint-propagation microbenchmark. LP relaxations and presolve can transform models before native solve; solver logs/versions aid interpretation but cannot make comparison fair when scopes differ. Configuration applicability is fixed by N before measurement to avoid post hoc engine selection. Runtime intersections may be smaller than coverage intersections due to TIMEOUT, ERROR, or missing timing. Each headline runtime identifies its group-level scope; do not extrapolate a small intersection to all puzzles of that N/source. A parallel profile may increase batch throughput while increasing per-worker latency; these are different objectives. Walltime includes scheduling/idle time and does not equal summed per-job solver time. Use only protocol-defined events; do not treat wall-clock throughput as solver runtime or rank algorithms when workload overlap is unbalanced.

M. A worked audit of a 4 x 4 Sudoku encoding

This appendix is a teaching example, not an added dataset instance or benchmark observation. It connects the general equations to concrete IDs, clauses and validation conditions. No solver is called, no runtime is measured, and solution uniqueness is not asserted. Counts below describe constructed formulas before preprocessing.

M.1. Input, illustrative solution and numbering

Set $n = 2, N = 4$. Use four diagonal clues and the following completed grid. Zero denotes an empty input cell; it is not an encoded value.

$$P = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad S = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 1 & 2 \\ 2 & 1 & 4 & 3 \\ 4 & 3 & 2 & 1 \end{pmatrix}.$$

There are 64 primary variables. With zero-based rows and columns and one-based values, the forward and inverse mappings are

$$q = \mathrm{id}(r, c, v) = 16r + 4c + v,$$
$$t = q - 1, \qquad r = \left\lfloor \frac{t}{16} \right\rfloor, \qquad c = \left\lfloor \frac{t \bmod 16}{4} \right\rfloor, \qquad v = (t \bmod 4) + 1.$$

In general, replace 16 by N^2 and 4 by N. For example, $q = 27$ gives $t = 26$, hence $(r, c, v) = (1, 2, 3)$; the forward map returns 27. Boundary IDs 1 and 64 represent $(0, 0, 1)$ and $(3, 3, 4)$. Zero, negative IDs and IDs above 64 lie outside the primary inverse mapping's domain.

Row-major clue traversal appends exactly four units: (1), (24), (44) and (61). For example, clue $(2, 2, 4)$ has ID $32 + 8 + 4 = 44$. The two clues valued 4 are compatible because they occupy different rows, columns and blocks.

M.2. Four group families with distinct meanings

The table follows the generator's literal order. Positions are zero-based within the 64-group sequence: 16 cell groups, 16 row–value groups, 16 column–value groups, then 16 block–value groups.

Position	Group	ID list
6	$(r, c) = (1, 2)$	$(25, 26, 27, 28)$
22	$(r, v) = (1, 3)$	$(19, 23, 27, 31)$
42	$(c, v) = (2, 3)$	$(11, 27, 43, 59)$
50	$(p, q, v) = (0, 0, 3)$	$(3, 7, 19, 23)$

The final example concerns value 3 in the top-left block, traversing cells $(0, 0), (0, 1), (1, 0), (1, 1)$. Position 6 contributes ALO $(25 \vee 26 \vee 27 \vee 28)$, while position 22 contributes $(19 \vee 23 \vee 27 \vee 31)$. Groups share primary variables: literal 27 participates in its cell, row, column and containing block constraints.

In S, cell $(1, 2)$ selects ID 25 rather than 27. Row 1 places value 3 at ID 19; column 2 places it at ID 11; the top-left block places it at ID 19. Each listed group therefore has exactly one true literal. This checks group interpretation directly, without solver inference.

M.3. Actual clauses for the first group

The first group is $Z = (1, 2, 3, 4)$, cell $(0, 0)$. Every encoding adds ALO $(1 \vee 2 \vee 3 \vee 4)$. Write $\overline{k}$ for the negation of ID k. The clauses below were checked through the existing CNF construction API with `top_id=64`; this is not a solver run.

Pairwise.

$$(\overline{1} \vee \overline{2}) \wedge (\overline{1} \vee \overline{3}) \wedge (\overline{1} \vee \overline{4}) \wedge (\overline{2} \vee \overline{3}) \wedge (\overline{2} \vee \overline{4}) \wedge (\overline{3} \vee \overline{4}).$$

There are six binary clauses, no auxiliaries, and the maximum allocated ID remains 64. Setting both 1 and 3 true immediately falsifies the second clause.

Sequential counter. The API allocates IDs 65, 66 and 67. Its actual clause order is

$$(\overline{1} \vee 65), \qquad (\overline{65} \vee 66),$$
$$(\overline{2} \vee \overline{65}), \qquad (\overline{2} \vee 66),$$
$$(\overline{66} \vee 67), \qquad (\overline{3} \vee \overline{66}),$$
$$(\overline{3} \vee 67), \qquad (\overline{4} \vee \overline{67}).$$

If only input 3 is true, choose $(65, 66, 67) = (0, 0, 1)$; all eight clauses hold. If both 1 and 3 are true, input 1 forces 65 and then 66 true, while input 3 forces 66 false. Clause print order does not change the CNF; the API's order need not match the conceptual presentation in the main text.

Bitwise. The selectors are 65 and 66. Here inputs 1, 2, 3 and 4 require codes 00, 01, 10, 11, respectively:

$(\overline{1} \vee \overline{65})$, $(\overline{1} \vee \overline{66})$,
$(\overline{2} \vee \overline{65})$, $(\overline{2} \vee 66)$,
$(\overline{3} \vee 65)$, $(\overline{3} \vee \overline{66})$,
$(\overline{4} \vee 65)$, $(\overline{4} \vee 66)$.

Inputs 2 and 3 simultaneously true require selector 65 to be both false and true. One true input fixes both bits. With all inputs false, every AMO clause holds for arbitrary selectors, but ALO fails. Thus AMO alone is not EXO.

M.4. Disjoint auxiliaries and model projection

Although the first group uses only IDs 1–4, all IDs 1–64 are reserved before auxiliary allocation. Starting auxiliaries at 5 would collide with the next cell's primary variables. Sequential uses auxiliary ranges 65 … 67 and 68 … 70 for the first two cells; bitwise uses 65 … 66 and 67 … 68. These ranges refer to separate encodings, not one combined formula.

After each family of 16 groups, sequential's `top_id` is 112, 160, 208, 256; bitwise's is 96, 128, 160, 192. Every new auxiliary exceeds the previous top, and primary IDs retain their meanings. The positive primary IDs decoding to S are

$$\{1, 6, 11, 16, 19, 24, 25, 30, 34, 37, 44, 47, 52, 55, 58, 61\}.$$

The decoder accepts this positive list as a partial model; it does not require all negative literals. Adding signed auxiliary IDs, such as 65 or -66, does not change the decoded grid. This does not certify that those auxiliary choices satisfy the CNF: decoding projects primary variables rather than checking a SAT model.

M.5. Exact counts for the four-clue example

There are $4N^2 = 64$ four-literal ALO clauses and four clue units. Pairwise contributes $64\binom{4}{2} = 384$ binary AMO clauses; each other encoding contributes $64 \cdot 8 = 512$. Duplicate clauses across groups are retained.

Encoding	Primary	Aux.	Variables	Clauses	Literals
pairwise	64	0	64	452	1028
seqcounter	64	192	256	580	1284
bitwise	64	128	192	580	1284

Literal counts are occurrences, not distinct variables: pairwise has $64 \cdot 4 + 384 \cdot 2 + 4 = 1028$. Equal clause totals at $N = 4$ do not imply identical formulas, propagation or solving time. Variable/clause totals were checked with `build_cnf`; literal totals follow from clause lengths and were checked against the constructed lists. These are structural checks of the example, not expanded experimental measurements.

M.6. Three validation layers and small failure cases

The parser checks input shape and domains; the decoder checks literal interpretation; the independent checker checks Sudoku. Their duties differ. Every row and column of S has set $\{1, 2, 3, 4\}$. Its top-left block has the same set, as do the other blocks. All four clues are preserved, so the checker accepts.

1. Removing ID 61 leaves cell $(3, 3)$ unassigned; the decoder rejects it. Adding ID 2 while retaining ID 1 selects two values in cell $(0, 0)$; the decoder rejects it. Adding -1 alongside 1 produces contradictory primary signs and is also rejected.
2. Changing only $S_{0,0}$ from 1 to 2 still gives a complete grid representable by a decodable positive list. Row 0 repeats 2 and omits 1, so the checker rejects it before checking clues.
3. Swapping the first two cells of row 0 preserves its value set but changes column 0 to $(2, 3, 2, 4)$, so column validation rejects it. To isolate a block failure, use the Latin square with rows 1234, 2341, 3412, 4123: all rows and columns pass, but the top-left block contains $1, 2, 2, 3$, so block validation rejects it.
4. Globally exchanging symbols 1 and 2 in S preserves all row, column and block conditions but changes clue $(0, 0)$. The checker reaches clue preservation and rejects the candidate there.
5. Replacing a value by Boolean `True` is invalid even though Python can compare it equal to 1: exact integer types are required. Conversely, conflicting but in-domain clues are structurally valid inputs; feasibility belongs to the model rather than the parser.

M.7. Audit pseudocode and implementation invariants

The following steps capture the implementation boundaries without introducing a new solving algorithm.

1. **Construct.** Set top $= N^3$ and start an empty clause list. For every group in the prescribed order, append ALO, call AMO using the current top, append its clauses and update top. Append row-major clue units last. After k groups there are exactly k ALO clauses; every literal's absolute ID is at most top.

2. **Decode.** Initialize a zero grid and empty sign map. Reject zero or non-integer literals. Ignore auxiliary IDs; for primary IDs require consistent signs and map each positive literal to its cell. Reject distinct values in one cell and reject any remaining zero cell at the end.
3. **Check grid.** Check shape, exact types/domains, every row, column and block value set, then clues, in that order. Publish a solution only after every check succeeds. The checker does not inspect the encoding, auxiliaries or solver conclusion, so an encoding/decoding error cannot certify itself as a Sudoku solution.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the Twenty-First International Joint Conference on Artificial Intelligence, pp. 399–404. URL: https://www.ijcai.org/Proceedings/09/Papers/074.pdf.

[2] Eén, N., Sörensson, N., 2004. An extensible SAT-solver, in: Theory and Applications of Satisfiability Testing, Springer. pp. 502–518. URL: https://doi.org/10.1007/978-3-540-24605-3_37, doi:10.1007/978-3-540-24605-3_37. proceedings of SAT 2003; publisher citation year 2004.

[3] Google OR-Tools Team, n.d. CP-SAT Solver. Google. URL: https://developers.google.com/optimization/cp/cp_solver. living documentation; accessed 16 October 2026. Recorded OR-Tools distribution: 9.15.6755.

[4] Gurobi Optimization, LLC, 2026. Gurobi Optimizer Reference Manual. URL: https://docs.gurobi.com/projects/optimizer/en/current/reference/python/model.html. year follows the vendor's explicit 2026 citation recommendation; accessed 16 October 2026. Recorded gurobipy and native runtime: 13.0.3.

[5] HiGHS Developers, n.d. HiGHS Documentation: Python Examples. URL: https://ergo-code.github.io/HiGHS/dev/interfaces/python/example-py/. living documentation; accessed 16 October 2026. The recorded local highspy runtime is 1.15.1.

[6] Huangfu, Q., Hall, J.A.J., 2018. Parallelizing the dual revised simplex method. Mathematical Programming Computation 10, 119–142. URL: https://doi.org/10.1007/s12532-017-0130-5, doi:10.1007/s12532-017-0130-5. preferred citation recommended by the HiGHS project; not a description of its full MIP algorithm.

[7] IBM, n.d.a. CPLEX Python API Reference Manual: Meet the Python API. URL: https://ibmdecisionoptimization.github.io/cplex-python-doc/manual/03_meet_python.html. documentation banner 22.1.2; accessed 16 October 2026. API reference only: the installed cplex distribution is 22.2.0.1 and recorded native runtime is 22.2.0.0; not DOcplex.MP.

[8] IBM, n.d.b. Creating a Constraint Programming Model (DOcplex.CP). URL: https://ibmdecisionoptimization.github.io/docplex-doc/cp/creating_model.html. documentation banner V2.32; accessed 16 October 2026. Recorded DOcplex interface: 2.32.264; CP Optimizer executable runtime: 22.2.0.0.

[9] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437. URL: https://doi.org/10.1007/978-3-319-94144-8_26, doi:10.1007/978-3-319-94144-8_26.

[10] Lynce, I., Ouaknine, J., 2006. Sudoku as a SAT problem. Author-hosted paper, listed in Proceedings of AIMATH 06. URL: https://people.mpi-sws.org/~joel/publications/sudoku05.pdf. 9-page author version; author landing page lists AIMATH 06 (2006), with copyright 2005.

[11] PySAT Developers, n.d.a. Cardinality Encodings (pysat.card). PySAT. URL: https://pysathq.github.io/docs/html/api/card.html. documentation banner 1.9.dev16; accessed 16 October 2026.

[12] PySAT Developers, n.d.b. PySAT cardinality encoder source: bitwise.hh. URL: https://github.com/pysathq/pysat/blob/218c1c163f837c6512d8970e4bdf49ae02cee779/cardenc/bitwise.hh. upstream commit 218c1c163f837c6512d8970e4bdf49ae02cee779; header credits the card library from BOLT; accessed 16 October 2026. Not yet matched to the installed wheel.

[13] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. URL: https://www.carstensinz.de/papers/CP-2005.pdf, doi:10.1007/11564751_73.

[14] Sudoku Exchange, n.d. Sudoku Exchange Puzzle Bank: hard.txt. GitHub repository grantm/sudoku-exchange-puzzle-bank. URL: https://github.com/grantm/sudoku-exchange-puzzle-bank/tree/d8c8ebaee0c08c412cfba96af1923dfa61c83317. commit d8c8ebaee0c08c412cfba96af1923dfa61c83317; dataset public-domain dedication; accessed 16 October 2026. Uniqueness is a source claim, not independently verified.